University of Essex | Online

Comparative Analysis of Deep Learning Approaches for Object Recognition

Using CIFAR – 10

Why Deep Learning for Image Recognition?

Convolutional Neural Networks

WHAT THEY DO

- Designed for image data
- Learn visual patterns, edges, shapes, textures directly from raw pixels
- Dominant approach in image classification
  Bishop and Bishop, 2024

TRADE-OFFS

- They require large amounts of training data
- Careful hyperparameter tuning is necessary
- Significant computational resources are needed
- Performance depends heavily on data quality and training design

THE EXPERIMENT

Three Models. One Test Set.

1. **Baseline CNN**
   Standard architecture
2. **Improved CNN**
   Data augmentation
3. **Transfer Learning**
   MobileNetV2 + fine-tuning

Dubai

UAE

Why this Problem Matters: Smart Traffic Systems

1

Adaptive Traffic Signals

ML and predictive analytics dynamically adjust signal timings using real-time sensor and IoT data to optimise traffic flow.

Digital Watch, 2025

2

Smart Monitoring Platforms

Computer vision analyses video feeds to detect vehicle types, traffic density, and road conditions for urban mobility planning.

Government of Dubai Media Office, 2025

3

Autonomous Vehicles

Robotaxis rely on deep learning combined with LiDAR, radar, and cameras for real-time object recognition and response.

Roads and Transport Authority, 2022; The UAE Government Platform, 2025

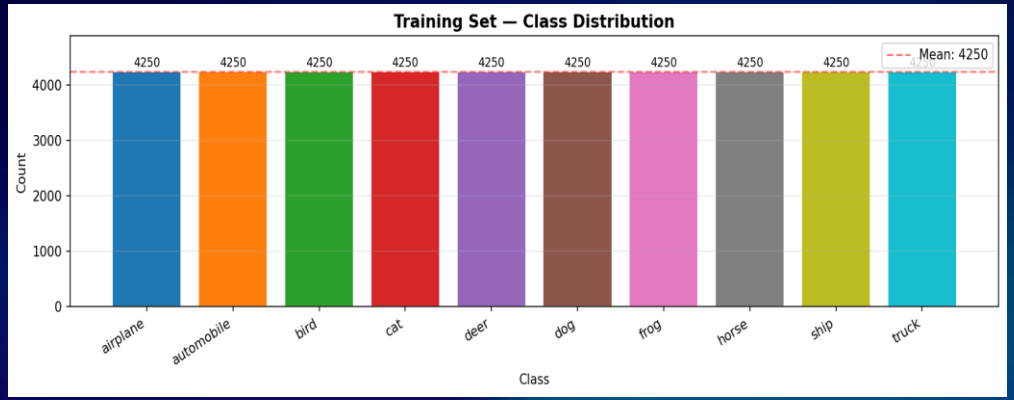
Understanding the CIFAR-10

- ▶ Dataset
- Model 1
- Model 2
- Model 3
- Comparison & Results

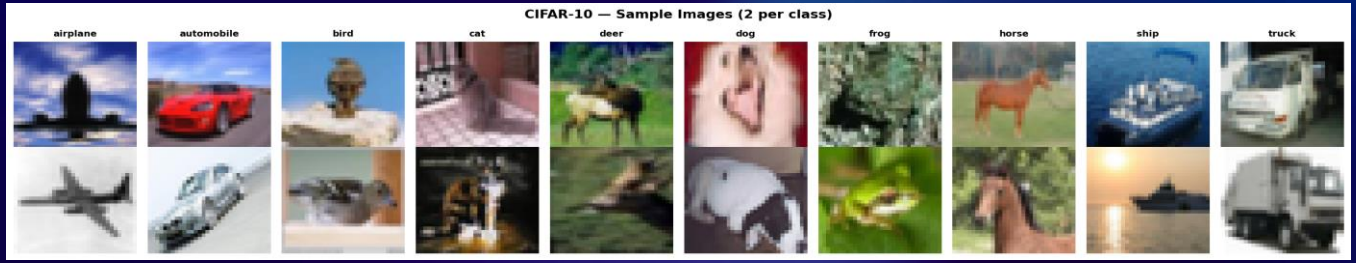
60,000
colour images

10
object classes

6,000
images per class



32 x 32 px – low resolution makes classification challenging



Because dataset is perfectly balanced, accuracy is a reliable and fair evaluation metric.

Preparing the Data for Reliable Training

▶ Dataset

Model 1

Model 2

Model 3

Comparison & Results

Original split

CIFAR-10 provides 50,000 training + 10,000 test images

Risk: two sets -> data leakage

Solution – three-way split. (Bishop and Bishop, 2024)

Training set
42,500 – 70.8%

Validation set
7,500 – 12.5%

Test set
10,000 – 16.7%

Stratified Split

Class proportions preserved

Pixel Normalisation

÷ 255 -> [0,1]

Baseline CNN: A Starting Point

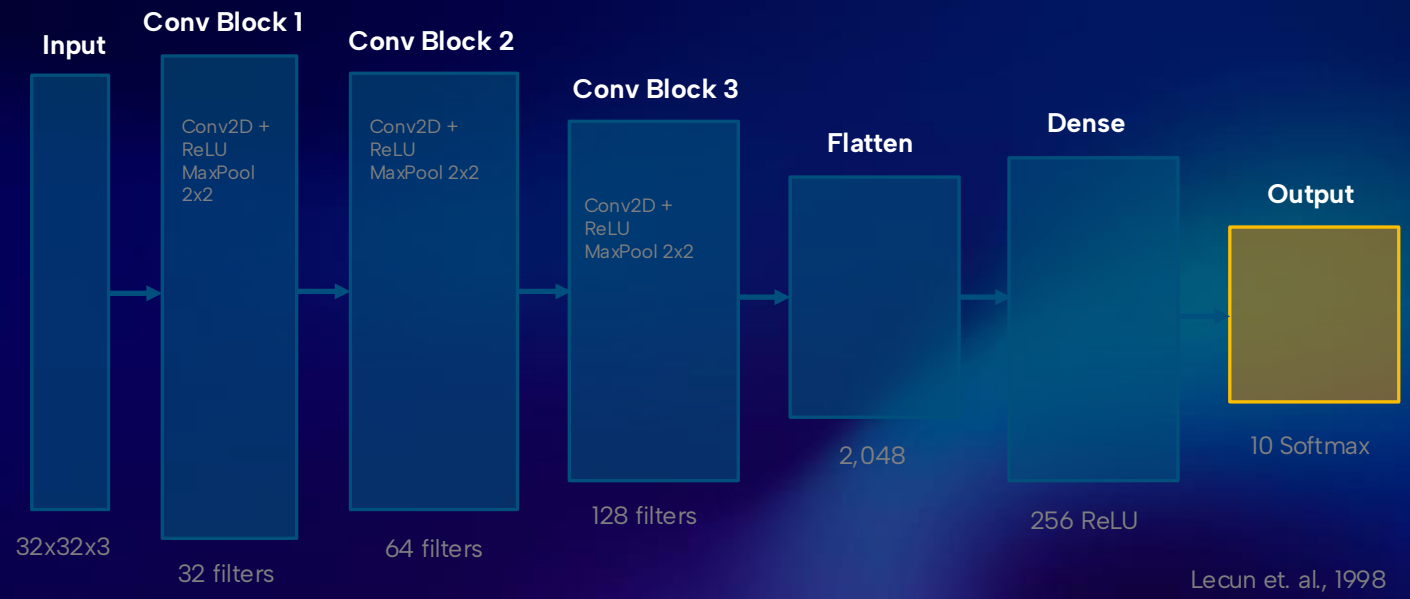
Dataset

▶ Model 1

Model 2

Model 3

Comparison & Results



Lecun et. al., 1998

620,362
trainable parameters

3
convolutional blocks

0
regularisation techniques

How the Baseline Model is Built

Dataset

▶ Model 1

Model 2

Model 3

Comparison & Results


```python
def build_baseline_cnn():
    model = keras.Sequential([
        keras.Input(shape=(32, 32, 3)),

        # Block 1 — detect basic features (edges, colours)
        layers.Conv2D(32, (3,3), padding='same', activation='relu'),
        layers.MaxPooling2D((2,2)),

        # Block 2 — detect more complex features
        layers.Conv2D(64, (3,3), padding='same', activation='relu'),
        layers.MaxPooling2D((2,2)),

        # Block 3 — detect high-level features
        layers.Conv2D(128, (3,3), padding='same', activation='relu'),
        layers.MaxPooling2D((2,2)),

        # Classifier
        layers.Flatten(),
        layers.Dense(256, activation='relu'),
        layers.Dense(NUM_CLASSES, activation='softmax')

    ], name='Baseline_CNN')
    return model
```

Keras Sequential API

builds the model as a linear stack of layers

padding="same"

preserves spatial dimensions

ReLU & Softmax

non-linearity for complex pattern learning (Sharma, 2017)

Training Setup: Baseline Model

Dataset

▶ Model 1

Model 2

Model 3

Comparison & Results

Compile Settings

Adam Optimiser

learning rate = 0.001

Standard default – efficient and stable convergence

Sparse categorical crossentropy

loss function

CIFAR-10 labels are integer-encoded

Training settings

64

batch size

50

max epochs

EarlyStopping

monitor	patience	action
val_loss	**8 epochs**	**restore best weights**

Halts training automatically when validation loss stops improving

```
baseline_cnn.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
callbacks=[
    EarlyStopping(monitor='val_loss', patience=8,
                  restore_best_weights=True, verbose=1),
    ModelCheckpoint('results/baseline_best.keras',
                    monitor='val_accuracy', save_best_only=True, verbose=0)
],
```

Baseline Performance: Signs of Overfitting

Dataset

▶ Model 1

Model 2

Model 3

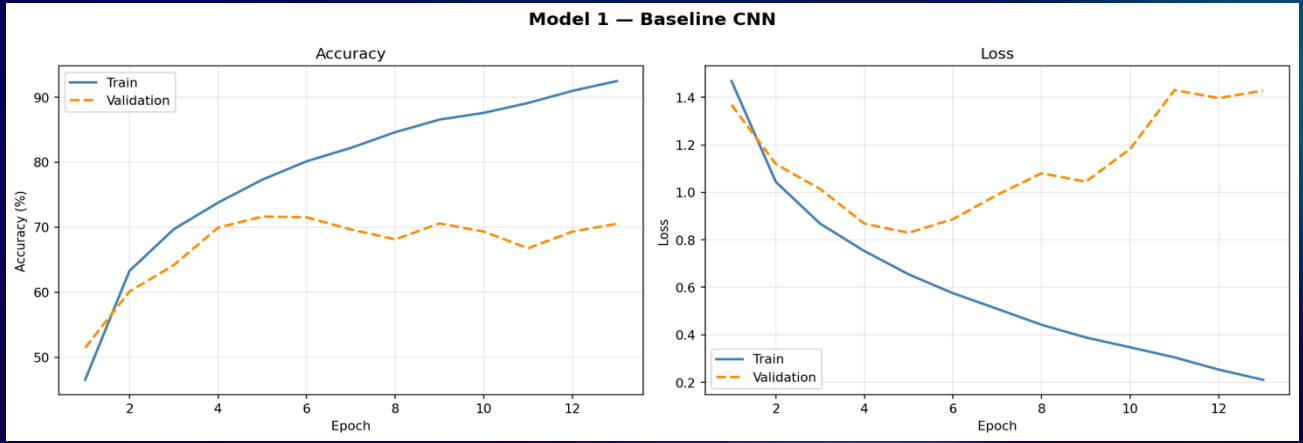
Comparison & Results

92.5%
train accuracy

vs

71.6%
val accuracy
(peak epoch 5)

20.9%
accuracy gap =
overfitting



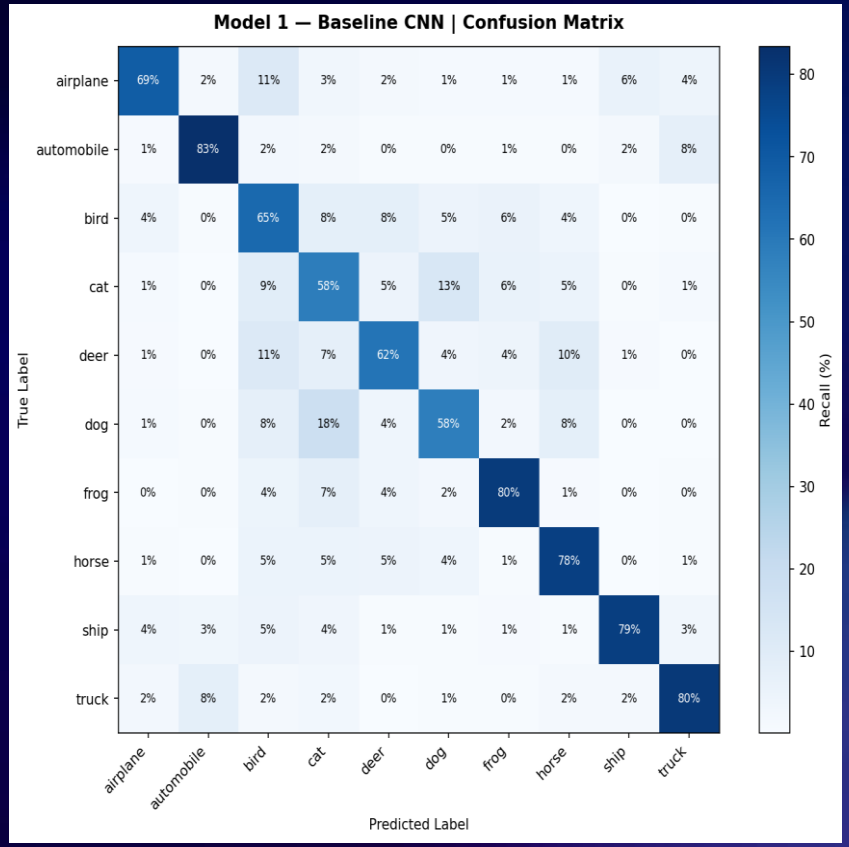
Why?

No regularisation. Flatten layer creates large parameter count – model memorises rather than generalises

EarlyStopping

Halted at epoch 13. Best weights from epoch 5 restored – preventing further degradation.

Where the Baseline Model Struggles



71.3%
test accuracy

71.5%
macro F1

Strongest classes

automobile	83%
frog	80%
truck	80%

Weakest classes

cat	58%	confused with dog (13%)
dog	58%	confused with cat (18%)
airplane	69%	confused with bird (11%)

Key Limitation

32x32 px fine textural detail is lost

Improving the CNN: Reducing Overfitting

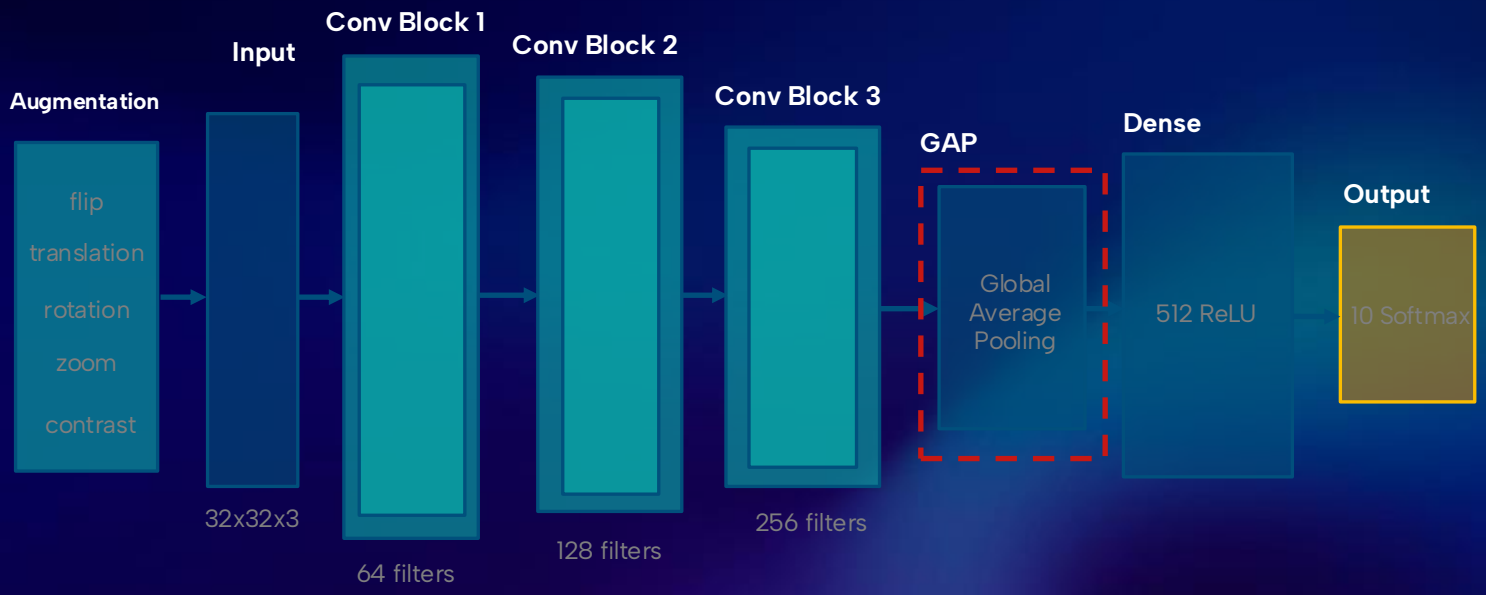
Dataset

Model 1

▶ Model 2

Model 3

Comparison & Results



Batch Normalisation

stabilises training gradients

Dropout

25%–45% per block – reduces overfitting

Global Avg Pooling

replaces Flattern – fewer parameters

L2 Regularisation

penalizes large weights across all conv and dense layers

Data Augmentation: Increasing Robustness

Regularisation technique – training data diversity by applying transformations to images (Shorten and Khoshgoftaar, 2019)

Dataset

Model 1

▶ Model 2

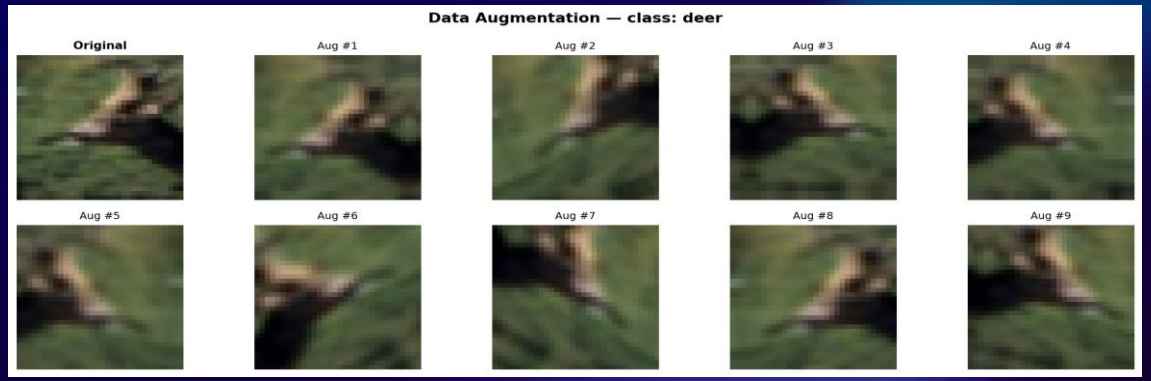
Model 3

Comparison & Results


```python
# — Augmentation layer ————————————————————
augmentation_layer = keras.Sequential([
    layers.RandomFlip('horizontal'),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
    layers.RandomContrast(0.1),
], name='augmentation')
```

5 TECHNIQUES

- Flip
- Translation
- Rotation
- Zoom
- Contrast



Architecture Enhancements for Generalisation

Dataset

Model 1

▶ Model 2

Model 3

Comparison & Results


```python
def build_improved_cnn():
    reg    = regularizers.l2(5e-4)
    inputs = keras.Input(shape=(32, 32, 3))

    # Augmentation — only runs when training=True
    x = augmentation_layer(inputs)

    # Block 1 — 64 filters
    x = layers.Conv2D(64, (3,3), padding='same', kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv2D(64, (3,3), padding='same', kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.MaxPooling2D((2,2))(x)
    x = layers.Dropout(0.25)(x)

    # Block 2 — 128 filters
    x = layers.Conv2D(128, (3,3), padding='same', kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv2D(128, (3,3), padding='same', kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.MaxPooling2D((2,2))(x)
    x = layers.Dropout(0.30)(x)

    # Block 3 — 256 filters
    x = layers.Conv2D(256, (3,3), padding='same', kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv2D(256, (3,3), padding='same', kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.MaxPooling2D((2,2))(x)
    x = layers.Dropout(0.35)(x)

    # Global Average Pooling — fewer params than Flatten
    x = layers.GlobalAveragePooling2D()(x)

    # Dense head
    x = layers.Dense(512, kernel_regularizer=reg)(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Dropout(0.45)(x)

    outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)
    return Model(inputs, outputs, name='Improved_CNN')
```

Augmentation Layer

Applied first – before feature extraction

Each training image randomly transformed

L2 Regularisation

Penalises large weights on all layers

Model is forced to learn simpler and more generalisable features

Batch Normalisation

Normalisation applied before the activation function

Stabilises gradient flow and speeds up training

Training Strategy: Improved Model

Same Adam optimiser & sparse categorical crossentropy as baseline – ensures a fair comparison

Dataset

Model 1

▶ Model 2

Model 3

Comparison & Results

ReduceLROnPlateau

monitor	patience	factor
val_loss	**5 epochs**	**0.5**

Halves learning rate when val_loss stalls – allows finer optimiser updates to continue improving

EarlyStopping

monitor	patience	action
val_loss	**10 epochs**	

Baseline patience: 8 -> Improved patience: 10

Longer patience accounts for augmentation variability

```
improved_cnn = build_improved_cnn()
improved_cnn.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
improved_cnn.summary()
print(f"\nTotal parameters: {improved_cnn.count_params():,}")
```

```
history_imp = improved_cnn.fit(
    x_train_norm, y_train,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS_IMP,
    validation_data=(x_val_norm, y_val),
    callbacks=[
        EarlyStopping(monitor='val_loss', patience=10,
                      restore_best_weights=True, verbose=1),
        ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                          patience=5, min_lr=1e-6, verbose=1),
        ModelCheckpoint('results/improved_best.keras',
                        monitor='val_accuracy', save_best_only=True, verbose=0)
    ],
    verbose=1
)
```

0.001	64	60	10
learning rate	batch size	max epochs	early stop patience

Improved Model: Learning Behaviour

Dataset

Model 1

▶ Model 2

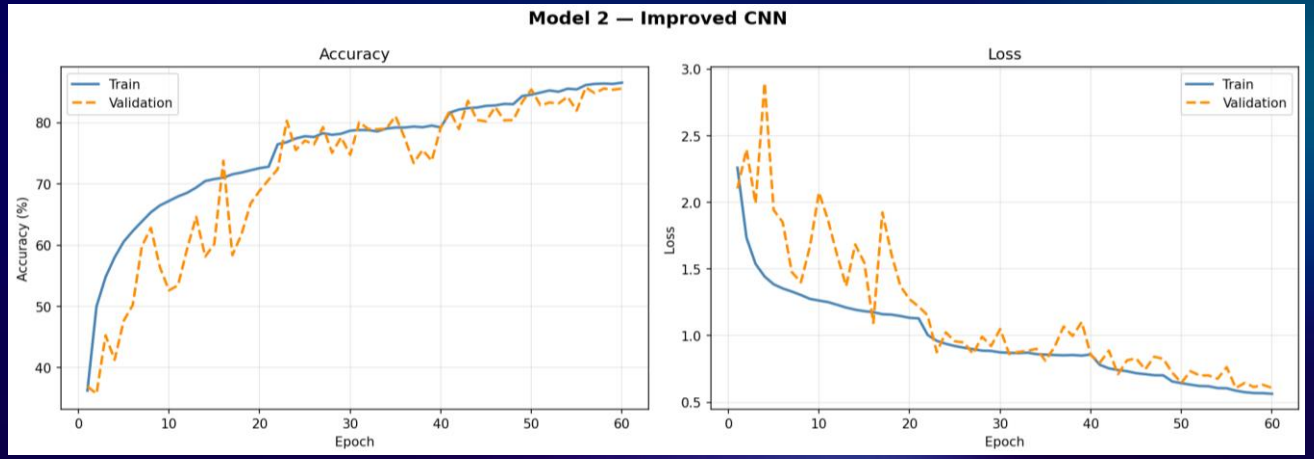
Model 3

Comparison & Results

86.6%
train accuracy

85.8%
val accuracy

60
epochs trained



Overfitting controlled

Train and val accuracy remain closely aligned throughout – regularisation is working.

Val loss function

Early fluctuations caused by augmentation – each epoch presents new variations. Stabilises over time.

Improved Model: Class-Level Performance

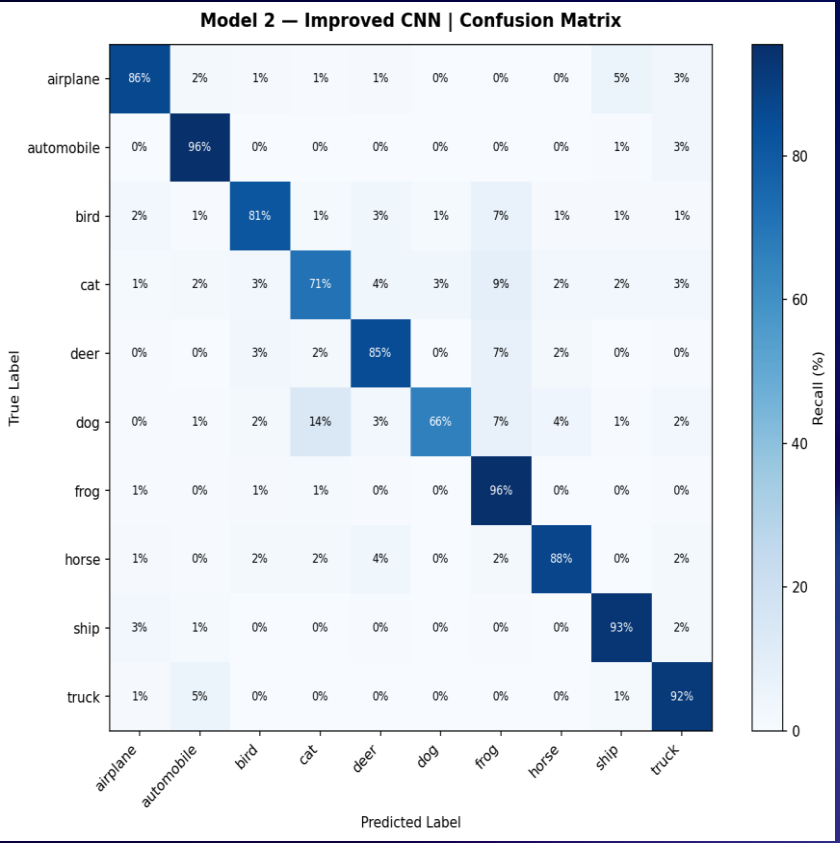
Dataset

Model 1

▶ Model 2

Model 3

Comparison & Results



85.3%	85.1%
test accuracy	macro F1

Strongest classes

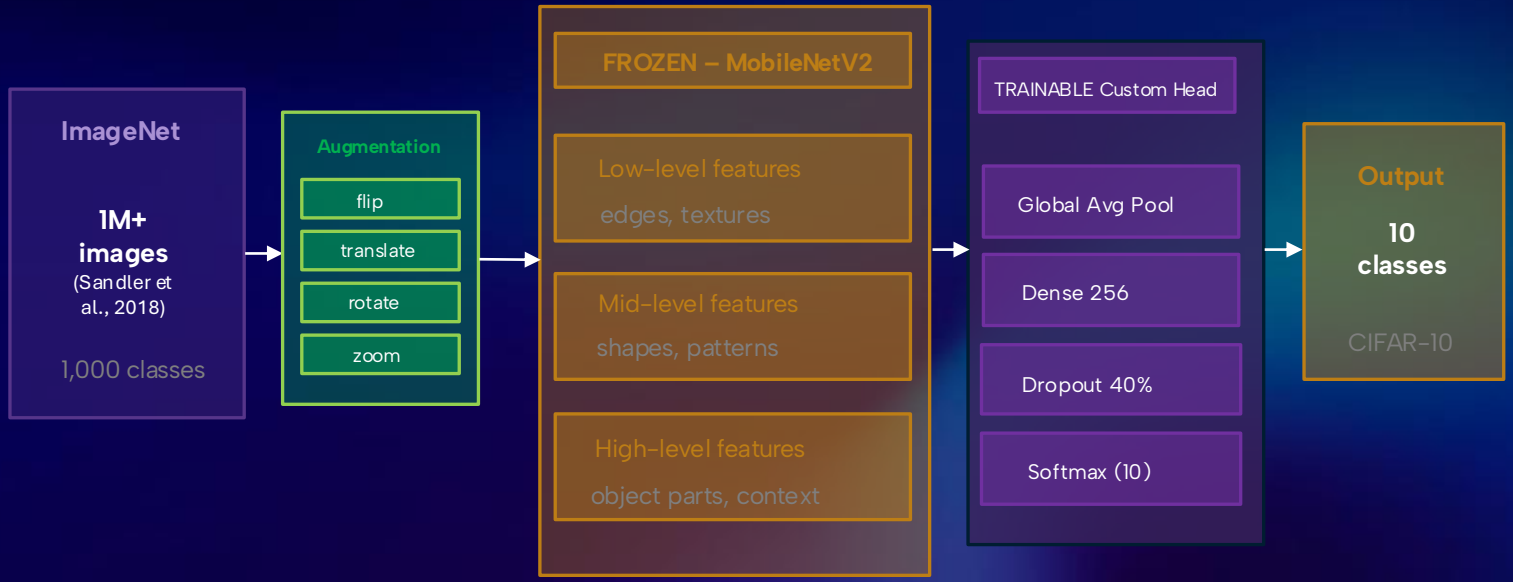
frog	96%
automobile	96%
ship	93%
truck	92%

Weakest classes

dog	66%	confused with cat (14%)
cat	71%	confused with frog (9%)

Transfer Learning with MobileNetV2

Uses a pre-trained model to leverage previously learned features rather than training from scratch (Pan and Yang, 2010)



2.6M

parameters after custom head added

2

training phases
Phase 1: head only
Phase 2: fin-tune base

Building the Transfer Learning Model

Dataset

Model 1

Model 2

▶ Model 3

Comparison & Results

```python
def build_mobilenet_model():
    inputs = keras.Input(shape=(TL_IMG_SIZE, TL_IMG_SIZE, 3))

    # Augmentation inside the model
    x = keras.Sequential([
        layers.RandomFlip('horizontal'),
        layers.RandomTranslation(0.1, 0.1),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.1),
    ], name='aug_tl')(inputs)

    # Pre-trained MobileNetV2 base — frozen for Phase 1
    base = MobileNetV2(
        include_top=False,
        weights='imagenet',
        input_tensor=x,
        alpha=1.0
    )
    base.trainable = False  # Phase 1: freeze

    # New classification head
    x = base.output
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dense(256, kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Dropout(0.4)(x)
    outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)

    model = Model(inputs=base.input, outputs=outputs, name='MobileNetV2_TL')
    return model, base
```

Augmentation inside model

Flip, translate, rotate, zoom – applied as first layer

Active during training only

include_top = False

Removes the original ImageNet classifier

Reuses only the feature extraction layers

weights="imagenet" + base frozen

Loads pre-trained weights. base_model.trainable=False locks all weights during Phase 1

alpha=1.0 retains full model capacity

Custom classification head

GAP -> Dense(256,L2) -> BN -> ReLU -> Dropout(0.5) -> Softmax

Adapts pre-trained features to CIFAR-10

Two-Phase Training Strategy

Dataset

Model 1

Model 2

▶ Model 3

Comparison & Results

Phase 1 Head training only

MobileNetV2Base – FROZEN

All weights locked
No gradient updates

Pre-trained ImageNet features preserved exactly as is

Custom Head – TRAINABLE

Dense 256 Dropout Softmax
Learns to map features to 10 classes

then →

Phase 2 Fine-tuning top layers

Lower layers – still FROZEN

Generic low-level features preserved edges, textures, gradients

Top layers – FINE-TUNED

Adapted to CIFAR-10 patterns
Small learning rate - gradual updates

Custom Head – continues training

Refines classification with updated features

Small learning rate during fine-tuning, ensures gradual updates that adapt the model without overwriting the valuable pre-trained features.

Training the Classification Head

0.001 learning rate Adam head only	**15** max epochs EarlyStopping triggered at 12
71.95% best val accuracy achieved at epoch 7	**5** ES patience val_loss monitored

```python
# — Phase 1: Feature Extraction ————————————————————————————————
tl_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("Transfer Learning — Phase 1: Feature Extraction")
print("MobileNetV2 base is frozen — training new head only")
print("-" * 50)

history_tl1 = tl_model.fit(
    train_tl_ds,
    epochs=15,
    validation_data=val_tl_ds,
    callbacks=[
        EarlyStopping(monitor='val_loss', patience=5,
                      restore_best_weights=True, verbose=1),
        ModelCheckpoint('results/tl_phase1_best.keras',
                        monitor='val_accuracy', save_best_only=True, verbose=0)
    ],
    verbose=1
)
```

EarlyStopping triggered

Stopped at epoch 12

Best weights from epoch 7 restored
patience = 5, monitor=val_loss

Why head-only first?

Trains the head to map pre-trained features to CIFAR-10 classes before any base weights are modified

Starting Phase 2

71.95% val accuracy established

Phase 2 fine-tuning builds on this
No augmentation – frozen base receives clean input

Fine-Tuning for Task Adaptation

Dataset

Model 1

Model 2

▶ **Model 3**

Comparison & Results

```python
# — Phase 2: Fine-tuning ————————————————————————————————————————
# Unfreeze only the last 30 layers of MobileNetV2
# Earlier layers detect simple universal features — no need to change them
# Later layers are more task-specific — fine-tuning helps here

mobilenet_base.trainable = True
FINE_TUNE_FROM = len(mobilenet_base.layers) - 30
for layer in mobilenet_base.layers[:FINE_TUNE_FROM]:
    layer.trainable = False

trainable_now = sum([tf.size(w).numpy() for w in tl_model.trainable_weights])
print(f"Trainable params after unfreeze: {trainable_now:,}")

# Very small LR — avoids overwriting pre-trained knowledge
tl_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

print("\nTransfer Learning — Phase 2: Fine-tuning")
print("Top 30 layers unfrozen — learning rate = 0.00001")
print("-" * 50)

history_tl2 = tl_model.fit(
    train_tl_ds,
    epochs=EPOCHS_TL,
    validation_data=val_tl_ds,
    callbacks=[
        EarlyStopping(monitor='val_loss', patience=8,
                      restore_best_weights=True, verbose=1),
        ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                          patience=4, min_lr=1e-7, verbose=1),
        ModelCheckpoint('results/tl_phase2_best.keras',
                        monitor='val_accuracy', save_best_only=True, verbose=0)
    ],
    verbose=1
)
```

0.00001

learning rate
100x smaller than Phase 1

30

epochs
full run — ES never triggered

Steady improvement

val accuracy climbed from 70.6% to 82.55% across all 30 epochs — still improving at epoch 30

82.55%

best val accuracy
epoch 30 — still improving

Callbacks not triggered

EarlyStopping and ReduceLROnPlateau never activated — training converged smoothly throughout

30

layers fine-tuned
top layers only

Transfer Learning: Training Dynamics

Dataset

Model 1

Model 2

▶ Model 3

Comparison & Results

~69%

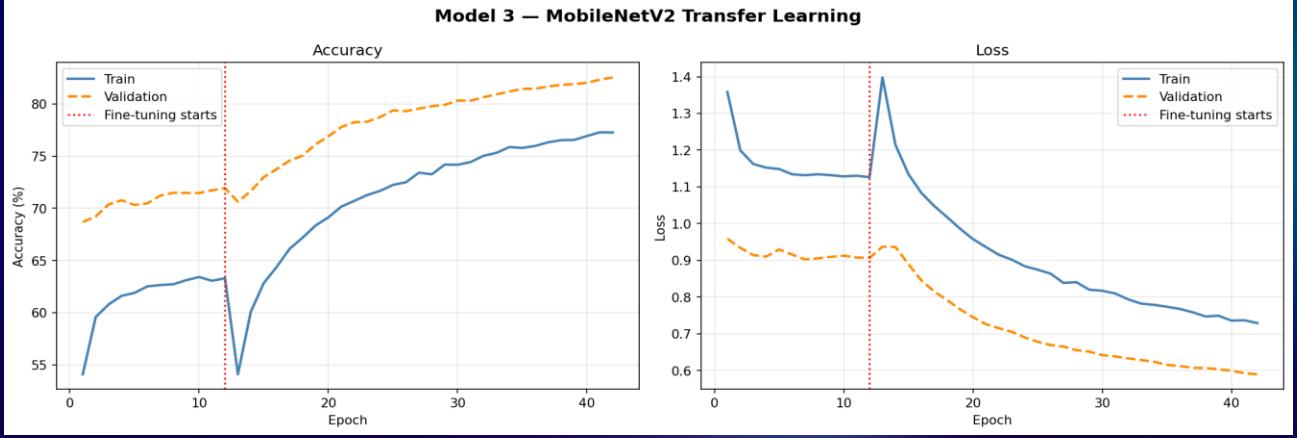
Phase 1 start val acc, pre-trained features

82.55%

final val accuracy, still improving at end

> val

val > train accuracy
strong generalisation



Phase 1 – head only

Val starts at ~69%, pre-trained features already capture useful visual patterns

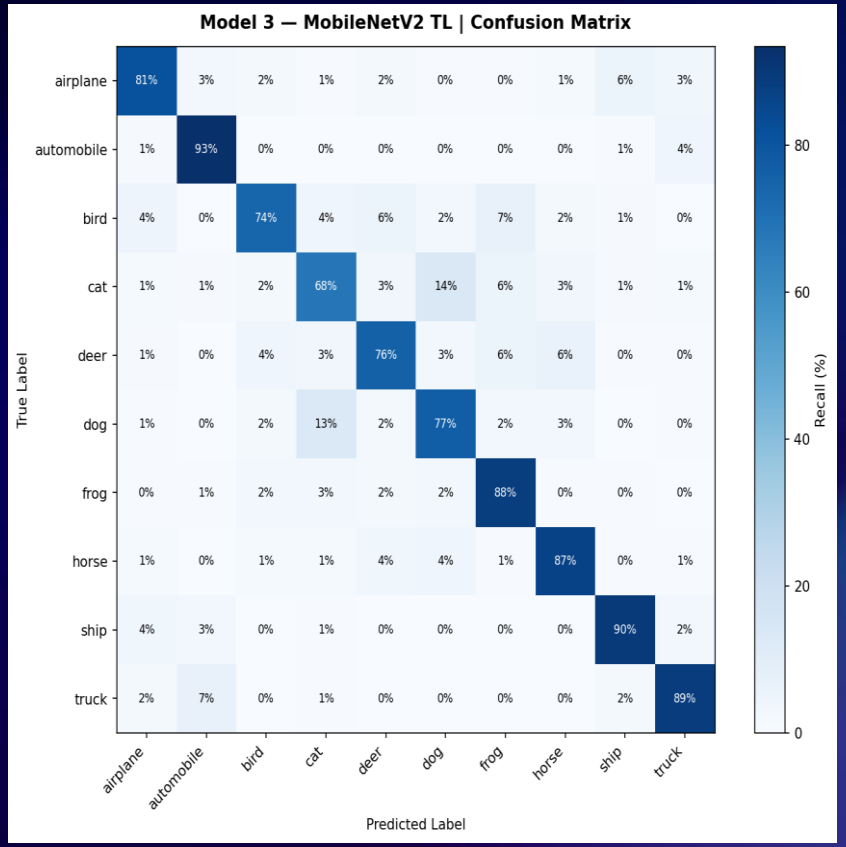
Phase transition drop

Brief accuracy drop and loss spike as frozen layers begin updating, expected and temporary

Phase 2 – steady climb

Both curves improve steadily to 82.55%. Upward trend at epoch 30 – more epochs could improve further

Transfer Learning: Class-Level Performance



82.3%	82.2%
test accuracy	macro F1

Strongest classes

automobile	93%
ship	90%
truck	89%
frog	88%
horse	87%

Weakest classes

cat	68%	confused with dog (14%)
bird	74%	confused with frog (7%)
dog	77%	confused with cat (13%)

Comparing Model Performance

Dataset

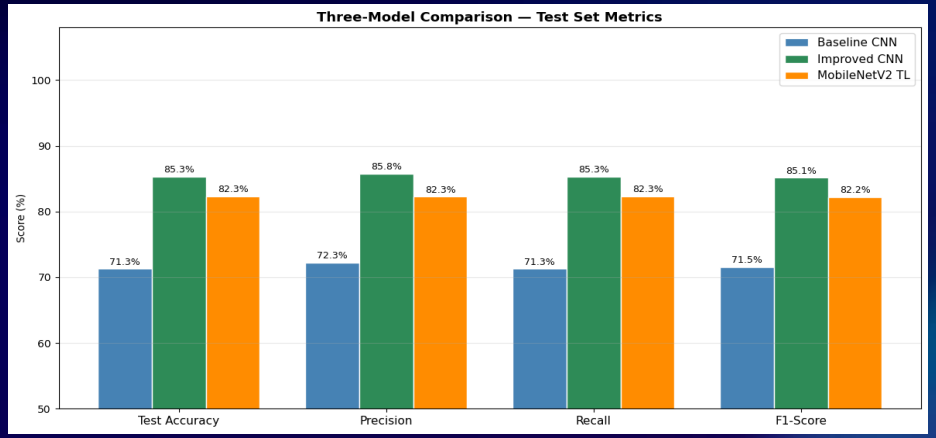
Model 1

Model 2

Model 3

▶ Comparison & Results

Consistent ranking across all four metrics – Improved CNN first, Transfer Learning second, Baseline third.



Model	Acc	F1
Baseline CNN	71.3%	71.5%
Improved CNN	85.3%	85.1%
TL	82.3%	82.2%
Improvement (1 -> 2)	+14pp	

Generalisation – train vs test gap

Baseline CNN

Train: 92.5% - Test: 71.3%

Gap: 21.2pp

Severe overfitting

Improved CNN

Train: 86.6% - Test: 85.3%

Gap: 1.3pp

Strong generalisation

Transfer Learning (TL)

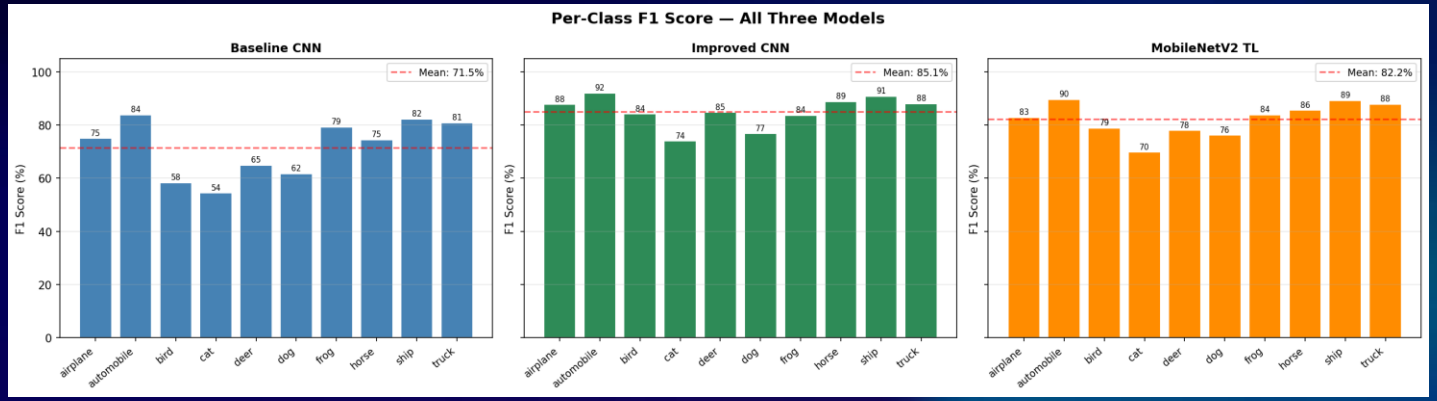
Train: 77.3% - Test: 82.3%

Gap: val > train

No overfitting

Where Models Perform Best and Worst

Cat and dog weakest across all three models – due to 32x32 resolution, not model quality



Weakest – all models

Cat (54/74/70%) and dog (62/77/76%) – visually ambiguous at 32x32 px.

Strongest – all models

Frog, automobile, ship – distinct shapes recognisable at low resolution

Improved CNN leads

Frog 84%, automobile 92%, ship 91% – highest in most classes

Baseline –widest spread

30pp gap: cat 54% to automobile 84% – uneven unregularised generalisation

Key Findings and Critical Insights

Dataset

Model 1

Model 2

Model 3

▶ Comparison & Results

Surprising fact

85.3%

Improved CNN test accuracy

vs

82.3%

MobileNetV2 TL test accuracy

-3pp – challenges the assumption that pre-trained always wins (Pan and Yang, 2010)

But the comparison is not entirely fair

1. **Resolution mismatch**

   MobileNetV2 designed for 224x224 – limited to 64x64 here. Reduced ability to extract detailed features

2. **Less training time**

   MobileNetV2 designed for 224x224 – limited to 64x64 here. Reduced ability to extract detailed features

3. **Domain gap**

   ImageNet vs CIFAR-10 are different domains. Improved CNN fully optimised for the target dataset

Under less constraint conditions – full resolution, more training, closer domain – transfer learning would likely close or surpass this gap

Strengths and Trade-Offs of Each Approach

	Baseline CNN 71.3%	Improved CNN 85.3%	Transfer Learning 82.3%
Strengths	• Simple and fast to train • Useful performance lower bound • Easy to understand and reproduce	• Full architectural control – tailored to the task • Improved performance and adaptability • Optimised end-to-end for CIFAR-10	• Leverages large-scale ImageNet knowledge • Effective in data-scarce scenarios • Efficient for edge device deployment
Limitations	• Prone to overfitting – no regularisation • Limited capacity – shallow architecture • Poor generalisation on visually similar classes	• Requires careful hyperparameter tuning • Dropout, LR and regularisation interact closely • Constrained by training data size and diversity	• Sensitive to input resolution and domain gap • ImageNet mismatch limits feature transfer • Requires additional fine-tuning for new domains

From Model to Real-World Deployment

1

Baseline CNN

Not suitable

Overfitting and low accuracy produce unreliable predictions- not safe for safety-critical traffic systems

2

Improved CNN

Limited

Strong accuracy but requires large labelled datasets and extended training – less practical for dynamic real-world conditions

3

Transfer Learning

Most suitable

Reduces dependence on labelled data. Designed for edge device execution – roadside cameras, embedded systems

Conclusion and Future Directions

Key Lessons

1 Regularisation is critical

Augmentation, BatchNorm, Dropout and L2 together drove a 14pp improvement. Model design is as important as model choice

2 Transfer Learning is context-dependent

Performance is influenced by input resolution, domain similarity and available compute – not just model architecture

3 Validation set is critical

Guided EarlyStopping and ReduceLROnPlateau decisions throughout – essential for reliable and unbiased model evaluation

Future Work

Higher resolution training

Train MobileNetV2 at 224x224 to align with its original design – expected to significantly close the performance gap with the Improved CNN

Hyperparameter optimisation

Apply grid search or Bayesian optimisation to systematically find optimal dropout, learning rate, and regularisation settings

Real-world traffic datasets

Evaluate all three approaches on real-world traffic surveillance data – essential step before practical deployment

References

Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Bishop, C.M. and Bishop, H. (2024). *Deep Learning*. Springer Nature.

Bjorck, J., Gomes, C., Selman, B. and Weinberger, K.Q. (2018). Understanding Batch Normalization. *arxiv.org*. [online] doi:https://doi.org/10.48550/arXiv.1806.02375.

Digital Watch (2025). *New AI traffic control promises faster commutes in Dubai | Digital Watch Observatory.* [online] Digital Watch Observatory. Available at: https://dig.watch/updates/new-ai-traffic-control-promises-faster-commutes-in-dubai [Accessed 5 Apr. 2026].

Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press.

Government of Dubai Media Office (2025). *RTA launches A Smart Digital Platform to monitor and analyse traffic in Dubai*. [online] Government of Dubai Media Office. Available at: https://mediaoffice.ae/en/news/2025/august/11-08/rta-launches-a-smart-digital-platform-to-monitor-and-analyse-traffic-in-dubai [Accessed 5 Apr. 2026].

Ioffe, S. and Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* [online] arXiv.org. Available at: https://arxiv.org/abs/1502.03167 [Accessed 4 Apr. 2026].

Krizhevsky, A. (2009). *CIFAR-10 and CIFAR-100 datasets*. [online] Toronto.edu. Available at: https://www.cs.toronto.edu/~kriz/cifar.html [Accessed 25 Mar. 2026].

Lecun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), pp.2278–2324. doi:https://doi.org/10.1109/5.726791.

Pan, S.J. and Yang, Q. (2010). A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering*, [online] 22(10), pp.1345–1359. doi:https://doi.org/10.1109/tkde.2009.191.

Pascanu, R., Mikolov, T. and Bengio, Y. (2013). On the difficulty of training Recurrent Neural Networks. *arXiv:1211.5063 [cs]*. [online] Available at: https://arxiv.org/abs/1211.5063 [Accessed 4 Apr. 2026].

Roads and Transport Authority (2022). *Self-driving transport*. [online] www.rta.ae. Available at: https://www.rta.ae/wps/portal/rta/ae/home/sdt [Accessed 5 Apr. 2026].

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. [online] arXiv.org. Available at: https://arxiv.org/abs/1801.04381 [Accessed 4 Apr. 2026].

Sharma , A. (2017). *Understanding Activation Functions in Neural Networks*. [online] Medium. Available at: https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0 [Accessed 4 Apr. 2026].

Shorten, C. and Khoshgoftaar, T.M. (2019). A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data*, 6(1). doi:https://doi.org/10.1186/s40537-019-0197-0.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting.*Journal of Machine Learning Research*, [online] 15(56), pp.1929–1958. Available at: https://jmlr.org/papers/v15/srivastava14a.html.

The UAE Government Platform (2025). *Use of autonomous vehicles | The Official Platform of the UAE Government*. [online] U.ae. Available at: https://u.ae/en/about-the-uae/digital-uae/regulatory-framework/Use-of-autonomous-vehicles [Accessed 5 Apr. 2026].

CREDITS: This presentation template was created by Slidesgo, and includes icons by Flaticon, and infographics & images by Unsplash, Freepik and Pexels.